Prior art — every Pokémon AI project we know of, and what each one means for us

Version 3.71.0 · Last updated 2026-08-07

Will, 2026-08-06: *"can you scour the internet for all similar or related projects, we have done this several times but i keep finding more."* **That recurrence is the problem this file exists to end.** The survey had been done at least three times in conversation and never written down, so every new link restarted it. This is the register; add to it rather than re-searching.

Everything here is sourced. Nothing is from memory. Where a figure comes from a video transcript rather than a paper or repository it says so, and it must be verified before it reaches the white paper. This project retracts its own numbers publicly; quoting somebody else's unverified is worse.

---

1. The field, at a glance

project	format	method	result	our format?
VGC-Bench (AAMAS 2026)	**VGC DOUBLES**	BC, MARL, LLM, heuristics + self-play / fictitious play / double oracle	**beat a professional** in single-team mirror	**YES**
Metamon (RLC 2025)	singles, gens 1–4	offline RL, large sequence models, no search	**top 10%** anonymous vs humans	no
Foul Play (pmariglia)	singles, most gens	root-parallelised **MCTS**, own engine, **eval function not rollouts**	top-100 ladder	**no — no mega support**
Future Sight AI	singles	search + ML, then **ML removed entirely**	avg 1547, **peak 1630** (top 5%)	not yet — *doubles announced*
PokéLLMon (2024)	singles	LLM, in-context RL, knowledge retrieval	49% ladder, 56% invited	no
Sarantinos (2023)	Gen 7 Random Battles	regret minimisation + usage priors	**33rd in the world**	no
PokeAgent Challenge (NeurIPS 2025)	two tracks	competition, 100+ teams	—	partly
poke-env	singles + doubles	Python/Gymnasium infrastructure	library, not an agent	infrastructure

The one-line read: exactly one project is playing our game, and it is the strongest one.

---

2. VGC-Bench — the only one in our format, and it is serious

Angliss, Cui, Hu, Rahman, **Peter Stone**. AAMAS 2026. arXiv 2506.10326 Title, read from the arXiv record 2026-09-09 (`export.arxiv.org/api/query?id_list=2506.10326`): *VGC-Bench: Towards Mastering Diverse Team Strategies in Competitive Pokémon*, Cameron Angliss, Jiaxun Cui, Jiaheng Hu, Arrasy Rahman, Peter Stone; first version 2025-06-12. This repository carried two OTHER subtitles for it (`docs/ABRA-whitepaper.md` ref. 5, `docs/SLOWKING-whitepaper.md` ref. 8); both were corrected in the same pass.

- **~10^139 team configurations** — the paper states this is larger than Chess, Go, Poker, StarCraft or Dota. That number is worth carrying: it is the best available answer to *why is this hard*.
- **700,000+ human battle logs**, open-sourced on GitHub and HuggingFace.

- Baselines across the whole spectrum: heuristics, LLMs, behaviour cloning, and MARL with **self-play, fictitious play and double oracle**.
- **In a single-team mirror match, their agent beats a professional VGC competitor.**
- **The finding that matters most to us:** as the number of teams grows, the best single-team algorithm gets *worse* and *more exploitable*, but generalises better to unseen teams. That is a named, measured trade-off between narrow optimisation and robustness — and it is exactly what WOBBUFFET measures on our side.

THE DATASET IS NOT USABLE BY US, AND THE CODE ALREADY KNEW THAT. This entry first claimed the opposite — that `fit_policy.js` had mis-sized the archive and we should re-examine it. Will: *"I THINK WE ALREADY LOOKED AT THEIRS AND IT DIDNT REALLY COVER REG MB / JUST A FEW DAYS."* He was right. Measured by downloading it:

```
their Reg M-B         324 bo1 + 3,843 bo3 = 4,167 games,  4 days, 2026-06-17 -> 06-20
our  bo3 store        9,701 games,                        15 days, 2026-07-23 -> today
overlap: 4,167 of 4,167 — 100% ALREADY IN OUR STORE, as data/games.ots.jsonl
```

`fit_policy.js` says *"4,167 games, 2026-06-18 -> 2026-06-21, 4 distinct days"*. Exactly right. The **700,000 figure is Reg M-A**, the previous regulation — 600 MB of a different metagame, no more current than the M-B slice. There is nothing to go and get.

The sharpest structural difference: we have features, they have a tensor

Will, 2026-08-06: *"SO DID VGC BENCH TRY A COLLINEARITY VARIABLE WHATEVER WE HAVE WITH MAG"* — no, and the reason is architectural rather than an omission on their part.

MAG is a conditional logit over 58 hand-named features. `P(pick j) = exp(w·x_j) / Σ exp(w·x_k)`, fitted by gradient ascent on 186,494 real human clicks. Every column is something a person chose to name. **VGC-Bench feeds a raw observation tensor to a neural network** (`vgc_bench/src/utils.py` defines the observation dimensions) and lets it learn its own representation.

Collinearity is a pathology of the first design only. When two named columns say nearly the same thing, the fit cannot attribute credit and may hand one a negative weight to cancel the other's overshoot. **That this HAPPENS in our model is established; every MAGNITUDE we had for it is withdrawn.**

> ***RETRACTED 2026-08-06.*** `data/collinearity-audit.json` *and* `data/policy-weights.json` *were both produced against an engine that has since been corrected. The weights are stamped* `2026-08-05T04:00:43Z`*; commit* `3be3f3b` *at* ***16:47 the same day*** *rewrote* `movesFirst` *in* `engine/board.js` *— "MAG stops re-deriving turn order" — because the old path carried* ***no dynamic speed at all****, so a Tailwind that speeds you up this turn was invisible. Nine further engine commits followed on 2026-08-06, including WIRES 123–132 and the whole accuracy family.* ***No weight, standard error, held-out likelihood or top-1 accuracy from that fit may be quoted.*** `engine/feature_fixture.js` *reports all 76 feature hashes UNCHANGED across that commit, including* `movesFirst`*.* ***The guard is blind, and this is the proof*** *— its 10 scenarios never construct a live speed inversion, so a rewrite of the feature it is watching moves no hash (ROADMAP #78).*

A net has nothing to audit because nothing was named. **The trade is interpretability against a diagnosis burden**, and both halves are real — we can argue about a weight and they cannot, and they have no collinearity problem and we do.

Statistical capacity is not why our set is 58. The corpus holds ~186k training decisions against 58 weights — roughly 3,000 per weight, against a conventional bar of 10–20. (Approximate on purpose: the decision count depends on legal-action enumeration, which is engine code.) The count is limited by SPEED —

every feature is recomputed per candidate, per rollout — not by data. See ROADMAP #79 for how the set actually came about, which is: four hand-written batches, no admission rule, no duplication check.

And MAG's objective is theirs. Fitting to predict a human click IS behaviour cloning; the function class differs and the target does not. It inherits BC's ceiling by construction — their own spread, BC alone winning **0.130** against BC-then-PPO, is the size of what sits above that ceiling.

The error was inferring coverage from a NAME. The HuggingFace file is called `logs_gen9championsvgc2026regmb.json`, which is our format's id, and it holds three days of it. Same shape as reading "Excadrillite" off a pattern or counting a player called `Victini_Emil` as a Victini. The rule that keeps being relearned: **open the file.**

---

3. Metamon — the strongest *result* without search

Grigsby, Xie, Sasek, Zheng, Zhu. RLC 2025. arXiv 2504.04395

Offline RL on a decade of human battles, reconstructing **first-person views from spectator logs** — the same problem our `durable-ingest.js` solves, and worth reading for how they did it. Large sequence models adapt to the opponent *from the trajectory alone*. **Top 10% anonymously against humans, with no explicit search**, beating both an LLM agent and a strong heuristic search engine.

Why it matters here: it is a direct counter-example to the assumption that search is required. MILTANK is a search. Before spending more on it, note that a no-search sequence model reached top 10% in singles.

---

4. Foul Play — closest in architecture, and it cannot play our format

pmariglia. repo · write-up

- **Root-parallelised MCTS** over many battle states, on a bespoke engine (**poke-engine**) — the same build-your-own-simulator decision as MEDICHAM, reached independently.
- **Guided by a custom evaluation function rather than rollouts.** That is precisely ROADMAP #24, in production, at top-100. Strong evidence the change is right.
- Moved *from* expectiminimax (~5 turns, every branch, ran out of clock) *to* MCTS (~10+ turns on promising branches) — the question ROADMAP #62 exists to answer, already answered by someone who shipped.
- **Damage rolls grouped by whether they cause a faint**, averaged within group, likelihoods summed (transcript). Cheaper *and* lower-variance; added as an arm to #24.
- **Chance nodes weighted by likelihood** — we sample uniformly (#32, #35).
- **Hidden info as a filtered possibility set**, narrowed as the battle reveals things — XATU picks one most-likely set instead.
- **Dynamax, mega evolution and Z-moves are NOT supported.** Champions is a mega format at 26% of usage, so **Foul Play structurally cannot play our format today.**

`poke-engine` **— the engine underneath it, and the closest thing to MEDICHAM that exists.** repo · crate

Rust, explicitly rewritten from Python *for search speed*. Generations 4–8, **singles only**, so we cannot use it. Its own README takes the same position ours does — *"This is not a perfect engine… nowhere near as complete or robust as the PokemonShowdown battle engine"* — with the difference that we are building a differential to quantify the gap and they are not.

The technique is worth taking. It takes a state and a move pair, generates every possible outcome, and returns **instructions that can be applied to AND REVERSED from** the state. That is make/unmake, as chess engines do it: the position is never copied, you mutate forward, search, then undo. `engine/rollout_leaf.js:455` calls `battleInit` **inside** the rollout loop, so we construct fresh instead — ~12,800 state constructions per decision at 200 rollouts × 64 pairs. **Whether that costs us anything is unmeasured**, and it is ROADMAP #77, scheduled behind the differential because optimising a moving target means re-optimising after every WIRE.

They also paid the language cost we have not: Python → Rust for speed, where we chose JavaScript because Showdown is JavaScript — the same reasoning Future Sight AI gives verbatim.

---

5. Future Sight AI — the cautionary one

Creator's own video account (transcript, unverified against code — the project is **closed source**).

- Win-probability model in **TensorFlow.js** on **2 million battles** donated by the Showdown team; every turn a separate example. **At most 81% accuracy**, which he states is near the ceiling randomness allows.
- Three further models predicting the opponent's next action; shipped publicly as the **Pokémon Battle Predictor** browser extension.
- Prunes the tree with those predictions — drops unlikely opponent moves, drops own moves with much worse outcomes, and applies **a probability floor below which a random event is not explored**. ROADMAP #37 is us not doing this.
- Modified Showdown's own simulator to fork at chance points; chose JavaScript *because the simulator is written in it*, rather than rewrite "a battle system that took almost a decade".
- 16 cores in the cloud → **just under 3 turns of lookahead in a 15-second limit.** Compare #61: MILTANK needs 26s against a 20s budget, on one core of sixteen.
- **Average 1547, peak 1630** — top 10% average, peak above the top 5%.
- **It plays to its opponent's level:** ~50% win rate against every rank, an unintended consequence of programming it to vary by opponent strength. He would have preferred it simply played better. **That is HYPNO's exact design risk** (#52, deferred by Will) and is evidence for keeping the opponent *model* separate from any modulation of our own play.
- **AND THEN HE REMOVED THE MACHINE LEARNING.** He wrote a structural method to get more from fewer explored turns, found it could guide the ML predictions, and then found it **outperformed the models on accuracy AND speed** — so the shipped agent uses no ML at all. PORYGON2, MAG and DODUO are our ML stack. This does not mean ours loses; it means *"the learned model obviously beats the heuristic"* is not a safe prior and the comparison has to be run.
- **Doubles is announced.** He says he kept it in mind throughout and the only obstacle is being one person. **Our "they play singles" positioning is a timing advantage, not a structural one.**

---

6. The rest of the field

PokéLLMon — Hu, Huang et al., arXiv 2402.01118. First LLM agent at human parity: in-context RL from battle feedback, knowledge retrieval against hallucination, and consistency constraints to stop "panic switching". 49% ladder / 56% invited. Its stated weakness is instructive — it loses to **attrition**, mishandling Toxic / Recover / Protect timing, which is a long-horizon planning failure rather than a knowledge failure.

Sarantinos, Cambridge, arXiv 2212.13338. Gen 7 Random Battles. **33rd in the world**, on 4 servers. Regret-minimisation opponent prediction; fills unknown properties from usage statistics; **explicitly rejects Monte Carlo sampling as too sample-inefficient** — which is what MILTANK's leaf does. Independently

names our #29: its first "biggest challenge" is keeping the team balanced, with a worked example that *is* THE SACK, and it asserts an agent cannot achieve it by MinMax or MCTS lookahead alone. Its ELO caveat is load-bearing for us: comparing human and AI ratings is invalid **unless they played each other** — laddering with ALAKAZAM is exactly that.

PokeAgent Challenge, NeurIPS 2025 — Karten, Grigsby et al., *The PokeAgent Challenge: Competitive and Long-Context Learning at Scale*, arXiv 2603.15563. The id is genuine: read from the arXiv record 2026-09-09 (`export.arxiv.org/api/query?id_list=2603.15563`), v2 dated 2026-03-16 — the write-up postdates the competition it reports, which is why a NeurIPS 2025 event carries a 2026 id. Two tracks, 100+ teams. **Not yet read.**

poke-env, repo — the Python/Gymnasium standard for Showdown agents, from École Polytechnique. Infrastructure rather than an agent; supports doubles. Most academic work above sits on it. We do not, because we are in JavaScript for the same reason Future Sight is.

VGC AI Competition (CoG 2025) and **pokemon-vgc-engine** (Simão Reis) — a *separate* VGC framework using randomly generated creatures rather than the real dex. Different problem; catalogued so it is not confused with VGC-Bench.

---

7. What this changes for us

1. **We are not first, and in our own format we are behind.** VGC-Bench beat a professional. Saying otherwise in the paper would be false and trivially checkable.
2. **The honest contribution is instrumentation, not policy.** Nobody above publishes a mechanics census that must be shown RED before it counts, a differential against the official engine, or ratchets on silent failure. Our retraction record is a feature: we publish what we withdrew.
3. **Three of our open tasks are confirmed by production systems** — evaluation function over rollouts (#24), likelihood-weighted chance nodes (#32/#35), policy-guided pruning (#37).
4. **One of our assumptions is challenged by two of them** — Metamon reached top 10% with no search, and Future Sight removed its ML entirely. Whatever we conclude, it must be measured.
5. **The fairness critique is coming.** A ladder bot is argued unfair rather than merely strong, because a human cannot search under a timer. Meet it deliberately.
6. **Two names are taken**: Future Sight, and Foul Play.